

School OS SaaS - Codex Environment & Engineering Handoff

Version: 0.2

Purpose: Human-readable and Codex-oriented handoff for VS Code development.

Status: Pre-database-design engineering baseline.

This guide summarizes the full project context. The detailed authoritative requirement set remains `docs/School_OS_SaaS_Master_Specification_v0.2.md`.

Codex Project Context - School OS SaaS v0.2

Mission

Build a commercially distributable, secure, configurable **School Operating System SaaS** for schools from Play Group/Nursery through Grade 12. The system must handle multiple campuses, retain historical records, support strict role/scope permissions, use approval workflows for sensitive changes, work offline where required, and remain extensible to future hardware and AI capabilities.

This is a long-term product. Shortcuts that make one screen easier but damage historical integrity, authorization, migrations, or future extensibility are unacceptable.

Deployment decision

The product uses a SaaS model where **each customer school gets its own Supabase project**. Multiple school projects may live under one Supabase organization controlled by the SaaS operator. A separate SaaS control plane can route school code -> school project and track customer/deployment/version metadata.

Operational school data must remain inside the school's project. The school project itself is the customer isolation boundary.

Inside one school project

One school may contain multiple campuses such as Main, Girls, Junior, Boys, etc. Each campus can have its own classes, sections, rooms, students, teachers, attendance, timetable, fees, exams, library, transport and other assignments. Higher authority users may view/manage multiple campuses according to permission; campus staff are scoped to assigned campuses.

Academic model

Schools define their own level structure. Examples include Play Group, Nursery, KG-1, KG-2, Grade 1 through Grade 12, but a school may begin at Nursery or use a different configured set.

Academic years have custom start/end dates and remain historically accessible. The system supports an active-year context while retaining previous years.

Classes may have multiple sections, room/capacity configuration, configurable roll-number sequencing, class teachers, subject teachers and timetables.

Student progression must use enrollment history. Never overwrite prior class/campus/section history when a student promotes, repeats or transfers campus.

People and accounts

Login supports email or admin-created username; school-generated identifiers are additional profile/login metadata as configured.

A human can have multiple roles under one account, e.g. teacher + parent. The system should distinguish the person/profile concept from authentication credentials.

Family/parent access is currently a shared family account model for father/mother/guardian acting as one parent role for linked children. One family account can access multiple children. The displayed responsible relationship depends on the family's situation.

Roles

Confirmed roles include Super Admin, School Owner, Principal, Vice Principal, Administrator, Accountant, HR, Teacher, Coordinator, Librarian, Transport Manager, Driver, Nurse/Medical Staff, Student, Parent/Family, Security Staff, Maintenance Staff, Exam Controller and similar configurable administrative roles. A dedicated IT/Admin role was considered but is not required as a fixed product role; permissions are configurable.

Roles alone do not determine access. Use granular permissions and scope.

Permissions and scope

Representative permissions: view, create, update, cancel/reverse, approve, reject, export, publish.

Representative scopes: all campuses, assigned campuses, class, section, subject, teaching assignment, own record.

Examples:

- Accountant manages finance-related operations but cannot freely modify finalized fees or unrelated attendance.
- Teacher sees own salary and teaching work, not general student fee records.
- Teacher marks attendance/marks only within authorized assignments/workflow.
- Principal has broad visibility, analytics and approval authority according to configured permission.
- Super Admin is powerful but still audited.
- Parents see only their linked children.
- Global search, AI and exports must respect permissions.

Student domain

Student profile may include student ID, admission/serial number, full name, photo, gender, DOB, nationality, national ID/B-Form, birth certificate, blood group, father/mother/guardian details, contacts, WhatsApp, address/city, previous school, enrollment date, admission class, status and other configured information.

Statuses include active, graduated, transferred, suspended, withdrawn, expelled, deceased and alumni. Status transitions require history, date, reason and actor where appropriate.

Student ID, internal UUID, admission number, serial number and roll number are conceptually distinct.

Admission numbers use school-configurable patterns such as school prefix + session/year + numeric sequence. Numeric sequence length may be configured up to eight digits.

Admissions

Possible stages: application, document verification, test, interview, approval, admission fee, enrollment, class allocation. Schools may disable/skip stages. Generate application number, admission number, student ID and dynamic documents such as admission form, offer/admission letter and student card.

Curriculum and teaching

Subjects/curriculum are school-configurable. Subjects can be compulsory, optional, elective, practical or custom. Different classes/years can use different curricula; individual students may choose optional subjects.

Teachers can teach multiple subjects, classes, sections and campuses. A subject can have a different teacher in each section. A section has one class teacher while different subjects may have different teachers.

Timetable

Support manual timetable management and eventual/initial automatic generation. Prevent teacher, class and room conflicts. Consider teacher availability, room availability, class availability, periods/week, school hours and breaks.

Attendance

Current implementation priority: manual and QR attendance.

Operational school requirement: attendance is primarily taken in the morning/first class, with configurable support for another attendance point after break. Do not assume subject-period attendance is the only or primary truth simply because period data exists.

Attendance states include present/absent/late/early departure/leave and related configured states. Reports include percentage, monthly/yearly views and parent notifications/absence alerts.

Attendance corrections require approval and history.

Future attendance devices

The core attendance model must not be tied to manual/QR capture. Future versions may integrate biometric/fingerprint terminals, RFID/NFC readers, cameras/face recognition and other devices. Use an adapter/event architecture so device capture ultimately produces canonical attendance records with source metadata. Do not store vendor-specific attendance as the only truth.

Homework/learning

Support homework, assignments, projects, worksheets, quizzes and online tests with due dates, attachments, marks, student submission, feedback, late status and parent visibility/notifications.

Exams/results

Support unlimited/custom assessment types per year: weekly/monthly tests, practice, homework/assignment assessments, term/midterm/final, annual, practical, oral, quiz and custom exams.

Assessment structures may differ by subject (theory/practical/custom components). Schools define grading ranges, grades, GPA/pass rules. Ranking can be class/section/grade/campus based or disabled.

Authorized users can enter marks, but Exam Controller review is the publication gate. Results flow through draft/submission/review/approval/publish/lock states. Post-publication corrections require approval and revision history.

Result cards/DMCs are generated dynamically with school branding, attendance, comments, signatures and grades.

Finance

Fee types include admission, tuition, exam, transport, library, computer, laboratory, sports, uniform, hostel, miscellaneous and custom types.

Fee structures vary by campus, class, student and academic year. Billing supports monthly, quarterly, half-yearly, annual, one-time and custom schedules.

Discounts include policy-based sibling/staff-child/scholarship/need-based rules and student-specific custom discounts. Recurring discounts are supported.

Payments support cash, bank and wallet evidence. A parent may submit amount, bank/wallet, transaction ID/date and screenshot; it remains pending until verified by authorized finance staff.

Support partial payment, annual/advance payment allocation, overpayment credit/refund/admin decision, overdue balance, late fee and reminders.

No destructive deletion of real financial history. Corrections are reversal/cancellation/adjustment operations with reason, approval and audit.

Invoices and receipts are generated on demand rather than stored automatically as PDFs. Financial records remain stored. Invoice numbering uses a permanent format such as INV-YYYY-MM-number. Verified payments have permanent receipt numbers.

HR/payroll/leave

Employee profiles include ID, photo, identification, qualification, specialization, experience, joining date, contract, salary, subject/class assignments, attendance, leave and documents.

Payroll is monthly. Components include salary, bonus, overtime and manual deduction with reason. Salary/increment behavior can be configured as automatic intervals or manual. Salary changes may require approval. Finalized payroll is locked and payslips are generated dynamically.

Leave types and approval chains are school-configurable for students and employees.

Library

Support books/types/copies, student and teacher borrowing, return/due/fine/history and a purchase/payment/status flow where relevant.

Transport

Initial scope: vehicles, drivers, routes, stops, pickup/drop-off, transport fee linkage, student assignment, bus attendance and parent notifications. Live GPS is future.

Hostel

Optional: hostel, building, floor, room, bed, student allocation, hostel fee, check-in/out, attendance, visitors.

Medical

Sensitive: blood group, allergies, emergency contacts, notes, vaccination, medication alerts, nurse visits, incidents. Medical data requires stricter authorization than normal student data.

Documents/storage

Uploaded source documents such as student photos, birth certificates and employee contracts are stored in structured private Supabase Storage. Current target: compress/validate to max 1 MB.

Generated certificates/ID cards/result cards/invoices/receipts/payslips are generated only when requested and normally are not automatically persisted as files.

Notifications and communication

Initial channels: in-app, push and email. WhatsApp is future. Notifications may be immediate, scheduled or event-triggered and have read/unread history.

Email initially may use app-password configuration, but provider abstraction should allow migration to Workspace/managed providers later.

AI

Planned AI includes teacher assistant, student tutor, homework assistant, question generation, result analysis, early warning, parent assistant and principal analytics. AI conversation history may be stored and administratively cleared according to policy.

AI may never bypass standard authorization. A teacher cannot obtain finance data through AI if the application would deny that teacher direct access.

Offline

Offline capability is required, especially attendance. Local sensitive data should be encrypted. Sync needs idempotency and conflict detection. Critical conflicts

such as contradictory attendance updates should surface for resolution rather than silently applying last-write-wins.

Audit

Audit sensitive actions including actor, action, timestamp, target, old/new where relevant, reason, approval and contextual metadata. Audit access itself is permission-protected. Super Admin actions are audited. Normal app users cannot silently erase audit history.

Backups/deployment

Need database backup, configuration backup and complete deployment-recreation procedures. The same source/migrations/storage/function setup must reproducibly deploy a new school Supabase project. Track schema/migration versions across schools.

Future scope

Deferred but architecture-relevant: biometric/fingerprint attendance, camera/face-recognition attendance, RFID/NFC, advanced QR anti-fraud, live GPS, WhatsApp, payment gateways, inventory/assets, IoT/smart classrooms, CCTV integrations, external/government reporting, accounting integrations, additional languages/RTL, voice, advanced predictive analytics and certificate-verification mechanisms.

Do not implement these prematurely. Preserve clean extension points.

Next step

Proceed to database/backend design in dependency order. Define foundation conventions, identity/RBAC/scope, school/campus/academic foundation, workflow/audit primitives, then domain tables. Every module design should explicitly document relationships, history, constraints, indexes, RLS, workflow hooks, audit events, storage, offline implications and migration order.

Question and Decision Map

This is a compact map of the product-discovery questions that produced the current architecture. The detailed wording and answers remain in `School_OS_SaaS_Master_Specification_v0.2.md`, especially Sections 73-75.

Initial product questions

Institution and academic structure

Questions: school/college scope, configurable grades, academic years, multiple campuses.

Decision: current product targets schools from Play Group/Nursery through Grade 12 with fully configurable levels. A school may omit Play Group or use its own structure. Multiple campuses are supported. Historical academic years are retained.

Users and roles

Questions: who logs in, what can each role do, whether roles need custom permissions.

Decision: broad school roles are supported, but authorization is not hard-coded solely by role. Use granular permissions, scope and approval authority.

Students and family

Questions: student fields, statuses, multiple children per parent, teachers who are also parents.

Decision: student profile is rich but should be relational. Families can link multiple students. One person/account may hold multiple roles. Student status history and enrollment history are required.

Admissions

Questions: application -> verification -> test -> interview -> approval -> fee -> enrollment -> class allocation.

Decision: configurable staged admission workflow; stages may be skipped. Generate school-facing numbers/documents dynamically.

Classes/sections/promotion

Questions: sections, room/capacity, automatic promotion, failed/repeat/manual cases.

Decision: classes/sections are configurable. Promotion creates a new enrollment record and preserves history.

Curriculum

Questions: fixed curriculum vs user-created, compulsory/optional/elective.

Decision: curriculum is school-configurable; students may have individual optional/elective choices.

Teachers

Questions: teacher profile, dashboard, class/subject assignments.

Decision: full employee/teacher profile; teachers can span multiple subjects/classes/sections/campuses; assignment context controls access.

Attendance

Questions: daily/subject/period attendance, late/leave/reporting, manual/QR/biometric future.

Decision: current operational truth is primarily morning/first-class attendance with optional post-break session. Manual and QR are initial capture methods. Corrections require approval. The architecture must support future biometric/camera/RFID/NFC adapters without redesigning canonical attendance.

Timetable

Questions: conflict prevention and generation.

Decision: prevent teacher/room/class conflicts; support automatic generation and manual adjustment.

Homework/learning

Questions: homework, projects, worksheets, quizzes, submissions, feedback.

Decision: support full learning-work lifecycle with student and parent visibility.

Exams/results

Questions: custom exam types, subject mark components, grades/GPA/ranking, result PDF.

Decision: flexible assessment engine. Exam Controller reviews/publishes. Published results lock; corrections use revision workflows.

Finance

Questions: fee types/frequencies, discounts, bank/wallet proof, partial/advance, overpayment, overdue, invoice/receipt storage.

Decision: configurable finance engine with ledger-like integrity. Financial facts persist; generated PDFs are on demand. Parent payment proof requires verification. No normal hard deletion of transactions.

Portals/communication

Questions: parent/student dashboards, notifications, messaging.

Decision: role-specific portals. In-app/push/email initially; WhatsApp later. Search and communication are permission-aware.

Library/transport/hostel/medical

Decision: supported as modular domains. GPS is future. Medical has stricter access.

HR/payroll/leave

Decision: full HR/employee management, monthly payroll, configurable salary increments/manual changes, bonus/overtime/deductions, configurable leave approvals.

Documents

Decision: source uploads go to private Storage, target ≤ 1 MB. Generated PDFs normally are not stored automatically.

AI

Decision: teacher/student/parent/principal AI features are planned. AI must respect normal authorization and cannot become a permission bypass.

Offline

Decision: required, especially for attendance. Local sensitive data is encrypted; important conflicts are explicitly resolved.

Deep architecture questions Q50-Q100

Q50-Q53 - authentication, multiple roles and parent login

- Login: email or admin-created username; retain school-generated IDs as metadata/identifiers.
- One account may have multiple roles, e.g. teacher + parent.
- Family currently uses a shared parent/guardian login acting as one family role for linked children.

Q54-Q57 - permissions and teaching scope

- Separate action permission from data scope.
- Teacher access is not automatically school-wide.
- Different sections can have different subject teachers.
- One section has a class teacher and multiple subject teachers.

Q58-Q66 - academic years, history, numbering, capacity and promotion

- Academic-year dates are configurable; history remains available.

- Important profile/status history is retained.
- Admission number format is school configurable.
- Roll numbering supports default or custom starting sequences.
- Capacity should be enforced with authorized override if designed.
- Promotion is a lifecycle process that creates new enrollment history.

Q67-Q72 - subjects/exams/publication

- Subject structures are configurable by class/year/student choice.
- Unlimited/custom exams are supported.
- Reusable templates are supported.
- Exam Controller is the publication gate after review.

Q73-Q77 - invoice/payment records

- Invoice PDF is not stored, but the underlying financial record and invoice number are persistent.
- Verified payment gets a permanent receipt number.
- Payment evidence can be approved/rejected/corrected.
- Duplicate transaction evidence should be detected.

Q78-Q80 - payroll/leave

- Payroll is monthly and locked after finalization.
- Salary change/increment may be configured automatically or manually rather than assuming one fixed salary-history rule.
- Leave approval hierarchy is school-configurable.

Q81-Q88 - storage, notifications, offline, backup

- Compress/validate uploads to ≤ 1 MB target.
- Structured Storage paths/buckets.
- Immediate/scheduled/event notifications.
- Email initially can use app-password style configuration but should be provider-abstracted.
- Offline conflict detection is preferred over silent overwrite.
- Local offline data is encrypted.
- Need database/configuration/deployment backup strategies.

Q89 - SaaS architecture evolution

The discussion temporarily considered a shared multi-school database. Final decision later changed to **Option A: one Supabase project per customer school** under a SaaS operation. This final choice supersedes earlier multi-tenant-table ideas.

Q90-Q94 - AI/search/audit/delete policy

- AI and global search obey normal permissions.
- AI conversation history can be stored and administratively deleted according to policy.
- Audit is access-controlled.
- Financial/results/attendance/audit histories use correction/reversal/history patterns rather than ordinary hard delete.

Q95-Q100 - senior roles, languages, product philosophy

- Owner/Super Admin/Principal capabilities are permission-configurable, not assumed solely by title.
- Super Admin is audited.
- English initially, architecture should allow localization later.
- Product philosophy selected: **Modern School Operating System** using the five core engines.

Final SaaS decision after Q100

The selected architecture is:

SaaS operator

- > Supabase Organization
 - > Control Plane Project
 - > School A Project
 - > School B Project
 - > School C Project

Each school project contains that school's Auth/Postgres/Storage/functions/policies. The application code and migration set are reusable. The school user should not need direct Supabase access for normal operations.

This final SaaS decision is the starting assumption for database design.

Decisions and Invariants

These are approved product/architecture decisions that Codex must treat as invariants until an explicit new decision supersedes them.

SaaS and tenancy

- One customer school uses one Supabase project.
- A separate SaaS control plane is allowed/recommended for routing, subscription, deployment and version metadata.

- Operational school data stays in the school project.
- One school project can contain multiple campuses.
- Campus is an authorization/operational scope, not a SaaS tenant.

Product architecture

- The product is a modern School Operating System, not isolated CRUD screens.
- Five core engines: Data, Permission, Workflow, Automation, AI.
- Audit, notifications, reporting, storage, offline and deployment/versioning are cross-cutting concerns.

Historical integrity

- Academic-year data is retained.
- Student campus/class/section/roll progression is modeled through enrollment history.
- Promotions create new historical enrollment state, not destructive overwrites.
- Student status history is preserved.
- Published results are locked; corrections are revision workflows.
- Attendance corrections preserve history and approval.
- Financial transactions are never normally hard-deleted.
- Finalized payroll is traceable/locked.

Identity and family

- Internal UUID and school-visible student/admission/roll identifiers are distinct concepts.
- One person/account may hold multiple roles, e.g. teacher + parent.
- Current parent model is a shared family login acting as one parent role for linked children.
- Parent access is limited to linked children.

Authorization

- Authorization is role + action + scope + contextual assignment + workflow state.
- Teacher access must derive from teaching/class responsibilities, not broad role name alone.
- Campus admin is limited to assigned campus unless additional scope is granted.
- Principal/Super Admin/Owner capabilities are permission-configured and audited.
- Global search, reports, exports, AI and offline data obey authorization.

Finance

- Finance data is immutable by correction/reversal pattern rather than destructive deletion.
- Parent bank/wallet evidence is not an official payment until verified.
- Duplicate transaction evidence should be detectable.
- Advance/partial/credit/refund/overpayment scenarios are first-class.
- Invoices/receipts are dynamically rendered; their financial facts and unique identifiers persist.

Exams/results

- Exam structures and grading are configurable.
- Exam Controller review/approval gates publication.
- Published results are locked.
- Mark corrections require a controlled revision flow.

Attendance

- Manual and QR are current priority capture methods.
- Operational attendance is primarily morning/first-class with optional post-break capture depending school setup.
- Corrections require approval.
- Canonical attendance must be capture-method independent.
- Future biometric/fingerprint, RFID/NFC, camera/face-recognition and other devices feed attendance through adapters/events; they do not replace the canonical attendance history with vendor-specific truth.

Documents/storage

- Important source uploads use private structured storage.
- Current upload target is ≤ 1 MB after compression/validation.
- Generated PDFs are usually on-demand and not automatically stored.

Client/offline

- Current primary PC target is Flutter Windows `.exe`.
- Offline support is required.
- Local sensitive data is encrypted.
- Critical sync conflicts are detected/resolved, not silently overwritten.

Deferred features

Deferred does not mean forgotten. Architecture should remain extensible for GPS, biometrics, cameras, RFID/NFC, WhatsApp, direct payment gateways, inventory, IoT, CCTV, government/accounting integrations, languages/RTL, voice and advanced predictive AI.

Feature Catalog and Status

Legend: **CORE** = intended core product; **OPTIONAL** = can be enabled/disabled; **FUTURE** = deferred; **CROSS-CUTTING** = applies to many modules.

Domain	Feature	Status	Important rule
SaaS	Control plane / school routing	CORE	Store customer/deployment metadata, not school operational records
SaaS	One Supabase project per customer school	CORE	Primary tenant isolation boundary
School	School profile/configuration	CORE	Configurable name/code/logo/address/currency/timezone/nu
School	Multiple campuses	CORE	Campus-scoped operations with senior cross-campus access
Academic	Academic years	CORE	Custom dates; historical access retained
Academic	Configurable levels/classes/sections/rooms/capacity	CORE	Do not hard-code Play Group -> Grade 12 list
Student	Student profile/status	CORE	Preserve important history
Student	Family/shared parent account	CORE	One family login may see multiple linked children
Student	Enrollment history	CORE	Year/campus/class/section/roll history is authoritative
Admission	Configurable application workflow	CORE	Verification/test/interview may be skipped

Domain	Feature	Status	Important rule
Academic	Curriculum/subjects/lectures	CORE	Class/year-specific and student optional subjects
Staff	Teacher/employee profiles	CORE	Teacher may also be parent
Academic	Teacher assignments	CORE	Different teacher per section/subject; multiple campuses possible
Timetable	Manual timetable	CORE	Prevent conflicts
Timetable	Automatic generation	CORE/ADVANCED	Manual adjustment remains possible
Attendance	Manual attendance	CORE	Morning/first-class primary pattern
Attendance	QR attendance	CORE	Same canonical attendance truth
Attendance	Correction approvals	CORE	History + reason + approver
Attendance	Biometric/fingerprint devices	FUTURE	Adapter/event into canonical attendance
Attendance	Camera/face recognition	FUTURE	Adapter/vision service + validation; privacy-heavy
Attendance	RFID/NFC	FUTURE	External token mapping + reader adapter
Learning	Homework/assignments/projects/workshops	CORE	Submission, marks, feedback, late state
Learning	Online tests	CORE/ADVANCED	MCQ/T-F/short/essay, timer, attempts
Exams	Custom assessment/exam types	CORE	Unlimited/custom per year
Exams	Grading/GPA/pass rules	CORE	School-defined

Domain	Feature	Status	Important rule
Results	Review/publish/lock/correction	CORE	Exam Controller gate
Finance	Fee	CORE	Campus/class/student/year configurable
Finance	Discounts/scholarships	CORE	Policy + custom/recurring
Finance	Partial/advance/overpayment	CORE	Allocation/credit/refund support
Finance	Bank/wallet evidence	CORE	Pending verification before official payment
Finance	Dynamic invoice/receipt	CORE	Do not auto-store PDFs
Finance	Direct payment gateway	FUTURE	Webhook/provider adapter
Parent	Parent portal	CORE	Linked children only
Student	Student portal	CORE	Personal academic information
HR	Employees/departments/designations/contracts	CORE	Configurable
Payroll	Monthly payroll	CORE	Bonus/overtime/manual deduction; lock final payroll
Leave	Student and employee leave	CORE	Configurable approval chains
Library	Catalog/borrow/return/loan	OPTIONAL	Students + teachers
Transport	Routes/vehicles/drivers/stops	OPTIONAL	GPS deferred
Transport	Live GPS	FUTURE	Provider adapter
Hostel	Buildings/rooms/beds/fallouts	OPTIONAL	Can be disabled
Medical	Student health/nurse incidents	OPTIONAL/SENSITIVE	Trigger permissions
Documents	Source document uploads	CORE	Private structured Storage, <=1 MB target
Documents	Dynamic certificates/cards/PDFs	CORE	Generate on demand
Notifications	In-app	CORE	Read/unread/history

Domain	Feature	Status	Important rule
Notifications	Push	CORE	Provider implementation TBD
Notifications	Email	CORE	App password initially; abstraction recommended
Notifications	WhatsApp	FUTURE	Provider/channel adapter
Calendar	Events/holidays/exams/PTM/etc.	CORE	Notifications supported
Reporting	PDF/Excel/CSV/print/custom reports	CORE	Permission-aware
Search	Global search	CORE	Permission-aware
Audit	Sensitive action audit	CROSS-CUTTING	Super Admin also audited
Workflow	Generic approval engine	CROSS-CUTTING	Typed workflows, configurable chains
Automation	Immediate/scheduled rules	CROSS-CUTTING	Avoid hard-coded notification spaghetti
Offline	Encrypted local cache and sync	CROSS-CUTTING	Conflict detection required
Backup	DB/config/deployment/recovery	CROSS-CUTTING	Restore rehearsal before GA
AI	Teacher assistant	CORE/ADVANCED	Permission-aware
AI	Student tutor/homework assistant	CORE/ADVANCED	Permission-aware
AI	Parent/principal assistants	CORE/ADVANCED	Permission-aware
AI	Predictive analytics/early warning	FUTURE/ADVANCED	Historical data quality + explainability
Operations	Inventory/assets	FUTURE	Large optional domain
Platform	IoT/smart classroom/CCTV	FUTURE	Provider-neutral integration only
Localization	English	CORE	Initial language
Localization	Urdu/Arabic/RTL/RTL-R	FUTURE	Keep UI localizable

SaaS and Database Guardrails

This document governs the upcoming Supabase/PostgreSQL design phase.

1. Deployment boundary

Each customer school has an independent Supabase project. Therefore the school project's schema should model one school and its campuses, not add tenant IDs everywhere by default.

A separate control-plane database may have tenants/customers because it serves the SaaS operator. Do not mix the control-plane customer model with school operational tables.

2. Database design principles

- Normalize by domain/history needs; do not create one giant student/employee/finance table.
- Use UUID primary keys for internal identity unless a strong reason exists otherwise.
- Keep school-facing identifiers separate and unique under the correct scope.
- Use foreign keys and database constraints for invariants that must always hold.
- Use check constraints or reference/configuration tables instead of unrestricted text where domain states matter.
- Preserve timestamps and actor information on sensitive lifecycle changes.
- Avoid deriving and storing volatile values such as current age when DOB is authoritative unless there is a documented reporting/cache reason.
- Separate authoritative facts from generated/derived representations.
- Use migrations for every schema change.
- Design indexes from actual query/scope patterns, not just foreign keys.
- Plan archival/partitioning only where justified by scale (audit/attendance/events may later qualify).

3. Historical modeling

Use history/lifecycle tables for facts that change over time and need reconstruction.

Examples:

- student enrollments per academic year
- status transitions
- campus moves
- teacher assignments

- fee obligations and adjustments
- payment allocations
- marks revisions
- attendance corrections
- salary/payroll periods
- approval state history where needed

A `current_*` convenience field or view may be added for performance/UI, but must not be the only historical truth.

4. Identity and auth

Supabase Auth account ID should not be overloaded as the business identity for every entity. Model person/profile and role/entity relationships explicitly so one human can be teacher + parent and so students/employees can exist before/without a login where business rules permit.

Design username login carefully because Supabase Auth is naturally email/password oriented; any username routing/alias approach must remain server-safe and migration-friendly.

5. Roles/permissions/scopes

Do not encode all authorization solely as fixed enums in application code.

Need a configurable model for:

- role
- permission/action
- role-permission grant
- user role assignment
- scope assignment / campus scope
- contextual assignments (teacher -> subject/class/section/campus)
- approval authority

RLS should be the final data-access boundary for client-accessible tables. Application UI hiding is not security.

6. Workflow/approval

The approval engine should have a shared core but typed domain behavior. Avoid an unstructured “everything in JSON” design that loses constraints. It should support request type, target, requester, reason, old/requested values, workflow status, approvers/steps, decision timestamps, and domain application result.

Some workflows need multi-stage approval; others may be single-stage or configurable.

7. Audit

Audit is not the same as domain history. Both may be needed.

- Domain history answers “what was the business state?”
- Audit answers “who did what, when, why, through which action/request?”

Do not make audit rows casually mutable/deletable from normal clients.

8. Finance

Model finance with ledger-like integrity. Do not represent payment as a boolean on a fee row.

At minimum distinguish:

- fee definition/type
- fee structure/rule
- student fee obligation/charge
- discounts/scholarships
- payment record
- payment evidence/submission
- allocation of payment to obligations/periods
- credit/refund
- reversal/cancellation/adjustment
- invoice number/render request context
- receipt number

Exact table names will be designed later.

9. Attendance

Canonical attendance must be independent from capture source.

Likely concepts to evaluate in design:

- attendance session (morning/post-break/etc.)
- student attendance status
- correction/revision
- capture source type
- optional device registry
- optional raw/imported source event
- source event -> identity resolution -> canonical attendance write
- synchronization/idempotency metadata

Do not prematurely store biometric templates or face embeddings in the core school database. Future biometric/vision designs need a separate privacy/security review.

10. Storage

Use private buckets/path conventions by domain/entity. Store meta-data/reference in Postgres; object bytes live in Storage. Enforce file type/size rules. Do not use public buckets for sensitive student/medical/contract documents.

11. Generated PDFs

Do not automatically persist every invoice, receipt, result card, payslip or certificate. Store the underlying record/version and generate the document when requested. If a future legal requirement needs immutable rendered copies, introduce it as a versioned feature rather than assuming it now.

12. Offline

Every offline-capable table should be evaluated for:

- stable IDs generated safely offline/online
- version/updated-at conflict detection
- idempotent writes
- tombstone/archive behavior
- encryption at rest locally
- sync scope minimization

Finance and finalized results require especially careful offline mutation rules; not every module must support full offline writes in the first implementation.

Security, Workflow and Audit Rules

Security posture

Use least privilege, default-deny for sensitive data, and server-enforced authorization. Never trust the Flutter UI to enforce critical permissions by itself.

Never expose to Flutter:

- Supabase service-role key
- database password
- Supabase management token
- SMTP/app password
- AI provider secrets
- payment gateway secrets
- future device vendor secrets

Permission model

A request should be evaluated against:

1. authenticated account
2. mapped person/business identity
3. active roles
4. action permission
5. scope/campus/class/section/subject ownership/assignment
6. target resource state
7. workflow restrictions

Examples of state restrictions:

- locked result cannot be edited directly
- finalized payroll cannot be edited directly
- verified financial records cannot be deleted
- historical attendance correction may require approval

Sensitive domains

Highest sensitivity includes medical records, payroll, financial records, identity documents, audit data and administrative secrets.

Medical access should be stricter than ordinary student-profile visibility.

Approval engine expectations

A change request needs enough information to be independently reviewed:

- request type
- target type and ID
- requester
- old value/state when applicable
- requested value/state
- reason
- evidence/attachments when applicable
- approval chain/step
- reviewer decision
- reviewer note
- timestamps
- application result/status

Approval chain is school-configurable. Example flows include Teacher -> Co-ordinator -> Principal, Teacher -> Principal, or direct/single-stage approval where school policy allows.

Audit expectations

Audit important security and business events such as:

- authentication/admin account changes
- role/permission/scope changes
- student identity/status changes
- attendance corrections
- mark/result changes and publication
- fee/discount/payment/reversal operations
- payroll/salary changes
- approval decisions
- document access/deletion where appropriate
- AI administrative deletion/retention actions
- backup/restore/deployment operations

Audit must itself be access-controlled. Super Admin is not exempt from auditing.

Deletion policy

- Financial history: reversal/cancellation, not normal hard delete.
- Published results: revision/correction, not destructive edit.
- Attendance history: correction trail.
- Audit: not deletable by normal role actions.
- AI conversations: admin may delete according to product policy, because these are not the same type of immutable business ledger.
- Uploaded documents: deletion rules depend on business/legal needs and should be audited where sensitive.

AI security

AI queries must go through permission-aware services/views and must not receive broad raw database access. Treat prompt content as untrusted input. Log/monitor access appropriately without storing secrets in prompts.

Future Architecture Roadmap

These features are **not current implementation commitments**. They are listed so today's architecture does not unnecessarily block them.

1. Attendance hardware ecosystem

Fingerprint / biometric terminals

Future requirement. Likely integration through vendor/device adapters. The canonical attendance service should accept validated identity/time events without depending on the vendor schema.

Do not decide now where biometric templates live. That requires a separate privacy, legal, retention and security design.

Camera / face-recognition attendance

Future high-complexity feature. Possible architecture:

camera/edge device -> recognition/anti-spoof service -> confidence
+ identity event -> validation policy -> attendance service

Potential requirements later:

- device registry
- camera location/campus mapping
- consent/legal policy
- retention policy for images/video
- face-template/model governance
- false positive/negative handling
- anti-spoofing
- human override/review
- confidence thresholds
- audit of automated attendance decisions

None of these should be implemented simply by adding a `face_vector` column to students.

RFID / NFC

Future medium-complexity feature. Student card/token mapping should be separable from student identity so cards can be issued/replaced/revoked.

Advanced QR anti-fraud

Possible future improvements: rotating/signed tokens, device confirmation, photo confirmation, token expiry/versioning.

2. Transport GPS

Keep vehicle/route/student-assignment core independent from GPS provider. Future telemetry should be provider-adapted and not required for basic transport operations.

3. Messaging channels

Notification engine should have channel/provider abstraction so WhatsApp or other future channels do not require changing domain events.

4. Payment gateways

Payment ledger/evidence model should accept future gateway webhook confirmations using idempotency and provider transaction references. Do not model today's bank screenshot workflow in a way that prevents automated payment confirmation later.

5. Inventory/assets

Large optional future domain covering procurement, assets, assignments, location, maintenance and disposal. Keep it modular rather than adding asset columns to unrelated tables.

6. IoT/smart classroom/CCTV

Treat as external device/event domains. Use device registry/integration adapters where necessary. CCTV integrations introduce substantial privacy/security requirements and should not be mixed into ordinary school records.

7. Government/accounting integrations

Prefer canonical internal models plus mapping/export/integration adapters. Do not reshape core student/finance data around a single external government's or accountant's schema.

8. Localization

English initially. Keep UI strings localizable and avoid using translated display text as database identifiers. Future Arabic/Urdu may require RTL layouts.

9. AI evolution

Future possibilities include voice interfaces, advanced tutoring, teaching-plan generation, predictive analytics and early-warning models. Preserve clean historical data and explicit authorization; do not make AI model output the authoritative business record without review/workflow where needed.

10. Certificate verification

Stable certificate/document identifiers can support future verification mechanisms. Blockchain is only one possible implementation and should not be assumed now.

Development and Change Process

Planning cadence

The current project-management baseline uses two-week sprints with architecture/release gates. The schedule is editable; architecture and security quality take priority over preserving an obsolete date.

Before coding a feature

Create/confirm:

- requirement reference
- module/domain owner
- actors and permissions
- scope rules
- states/lifecycle
- source-of-truth data
- history requirements
- approval/correction behavior
- audit behavior
- notifications/automations
- storage/files
- offline behavior
- reports/search/AI visibility
- migration dependencies
- acceptance tests

Architecture Decision Records (ADR)

Use an ADR when a decision changes or fixes an important technical direction: tenant model, identity/auth strategy, finance ledger model, workflow engine design, offline conflict strategy, device integration model, etc.

Do not rewrite old ADR history. Mark superseded decisions and link to the replacement.

Database migrations

- One logical change per migration where practical.
- Migrations must be repeatable in a fresh school project.
- Validate upgrade from previous schema where releases already exist.
- Never rely on “I clicked this in the Supabase dashboard” as the only setup step.
- Include functions/triggers/RLS/storage-policy setup in versioned deployment material.
- Keep seed/default configuration separate from customer operational data.

Testing

At minimum test:

- happy path
- invalid state transition
- unauthorized user
- wrong campus/scope
- historical record behavior
- approval-required behavior
- duplicate/idempotency behavior where relevant
- generated report/document correctness
- migration/install behavior
- offline conflict behavior for offline-capable modules

Security/RLS tests are release blockers for sensitive modules.

Change control

When a new requirement appears:

1. Record it.
2. Classify CORE / OPTIONAL / FUTURE.
3. Evaluate impact on existing invariants and schema.
4. Decide whether an extension point is needed now.
5. Update requirements version and ADR/change log if architectural.
6. Re-estimate project plan if scope enters a current release.

Do not turn every future idea into immediate schema complexity. Conversely, do not create rigid schema that obviously forces destructive redesign for already-known future directions.

Definition of Done for a backend/domain change

- requirement implemented
 - schema migration versioned
 - constraints/indexes reviewed
 - RLS/permissions implemented and negative-tested
 - workflow/audit hooks implemented where needed
 - storage policy implemented where needed
 - automated tests pass
 - offline implications addressed
 - deployment from clean project tested for foundational changes
 - documentation/ADR updated
-

Database Design - Next Phase Plan

There are no remaining product questions that block beginning database design. Some technical details remain intentionally TBD and will be resolved during design with ADRs.

Phase 1 - foundation conventions

Before creating domain tables, decide and document:

- schema namespaces (**public** vs dedicated schemas where useful)
- UUID generation convention
- timestamps (**created_at**, **updated_at**, event times)
- actor attribution convention
- soft delete/archive conventions
- status/reference/configuration strategy
- migration naming/versioning
- RLS helper function strategy
- audit architecture
- approval/workflow architecture
- Storage bucket/path conventions
- Supabase Auth mapping strategy
- local/offline IDs and sync/version metadata

Phase 2 - identity and authorization

Design:

- person/profile
- auth account mapping
- username/email login alias/routing mechanism
- roles
- permissions
- role-permission grants
- user-role assignments
- campus/scope grants
- contextual teacher/class/subject assignments
- approval authority

This must be robust before large business modules.

Phase 3 - school and academic foundation

Design school settings, campuses, academic years, academic levels/classes, sections, rooms, roll/numbering configuration and module settings.

Phase 4 - generic platform engines

Design enough of:

- approval/workflow core
- audit core
- notification event/inbox core
- automation hooks
- storage metadata

to avoid reimplementing them differently in each module.

Phase 5 - people/student/staff domains

Design student core, family relationships, admissions, enrollment history, status history, employee/teacher profiles and teaching assignments.

Phase 6 - academic operations

Curriculum, timetable, attendance, learning work, exams/results.

Attendance design must include future capture-source extensibility without implementing biometric/camera systems now.

Phase 7 - finance and administration

Fees, discounts, obligations, invoices, payments, allocations, credits/refunds/reversals, HR/payroll/leave.

Phase 8 - optional domains

Library, transport, hostel, medical, etc.

Phase 9 - reporting/search/offline/AI integration

Design read models/views/indexes, offline sync metadata, analytics and permission-aware AI service interfaces.

Deliverable expected for each table/entity

Use `templates/DATABASE_ENTITY_DESIGN_TEMPLATE.md`. Every entity proposal must include:

- purpose
- authoritative/non-authoritative status
- fields/data types
- primary/unique keys
- foreign keys
- lifecycle/status

- history strategy
- constraints
- indexes
- RLS rules
- workflow/approval hooks
- audit events
- storage links
- offline/sync needs
- deletion/retention
- example queries
- migration/dependency order

First database-design workshop

Start with a **Foundation ERD/module dependency draft**, not SQL. Freeze entity boundaries first, then write migrations in batches.

Recommended Repository Structure

This structure is a starting convention for Codex and developers. It may evolve through ADRs.

```

school-os/
+-- AGENTS.md
+-- CODEX_PROJECT_CONTEXT.md
+-- README.md
+-- docs/
|   +-- School_OS_SaaS_Master_Specification_v0.2.md
|   +-- DECISIONS_AND_INVARIANTS.md
|   +-- QUESTION_AND_DECISION_MAP.md
|   +-- FEATURE_CATALOG_AND_STATUS.md
|   +-- SAAS_AND_DATABASE_GUARDRAILS.md
|   +-- SECURITY_WORKFLOW_AND_AUDIT.md
|   +-- FUTURE_ARCHITECTURE_ROADMAP.md
|   +-- DEVELOPMENT_AND_CHANGE_PROCESS.md
|   +-- DATABASE_DESIGN_NEXT_PHASE.md
|   +-- REPOSITORY_STRUCTURE.md
|   +-- ARTIFACT_INDEX.md
|   `-- adr/
+-- templates/
|   +-- ADR_TEMPLATE.md
|   `-- DATABASE_ENTITY_DESIGN_TEMPLATE.md
+-- flutter_app/
|   +-- lib/

```

```

| | +-- app/
| | +-- core/
| | +-- features/
| | +-- services/
| | `-- shared/
| `-- test/
+-- supabase/
| +-- migrations/
| +-- functions/
| +-- seed/
| +-- config/
| `-- tests/
+-- control_plane/
+-- scripts/
| +-- bootstrap_school/
| +-- migration_checks/
| `-- backups/
+-- tests/
| +-- integration/
| +-- security/
| `-- deployment/
`-- reference/

```

Boundaries

- `flutter_app` should not contain secrets or be the only enforcement point for business security.
- `supabase/migrations` is the reproducible source of schema truth.
- `supabase/functions` contains server-only operations and integrations when appropriate.
- `control_plane` remains separate from a school operational project.
- `docs/adr` records architectural decisions.
- `reference` contains large human-facing artifacts; Markdown in `docs` remains the machine-readable authority.

Artifact Index

Authoritative machine-readable/project context

- `../AGENTS.md` - Codex behavior and non-negotiable architecture rules.
- `../CODEX_PROJECT_CONTEXT.md` - complete session handoff summary.
- `School_OS_SaaS_Master_Specification_v0.2.md` - full requirements, Q&A decisions and original product architecture.
- `DECISIONS_AND_INVARIANTS.md` - invariants.

- `FEATURE_CATALOG_AND_STATUS.md` - feature status matrix.
- `SAAS_AND_DATABASE_GUARDRAILS.md` - database design rules.
- `SECURITY_WORKFLOW_AND_AUDIT.md` - security/workflow/audit rules.
- `FUTURE_ARCHITECTURE_ROADMAP.md` - future extensibility.
- `DATABASE_DESIGN_NEXT_PHASE.md` - upcoming work sequence.

Human presentation/reference artifacts in reference/

- `School_OS_SaaS_Master_Specification_v0.1.pdf` - original 64-page requirements baseline snapshot.
- `School_OS_SaaS_Master_Specification_v0.1.docx` - editable original baseline snapshot.
- `School_OS_SaaS_Project_Management_Handbook_v0.1.pdf` - project-management execution handbook.
- `School_OS_SaaS_Project_Management_Handbook_v0.1.docx` - editable PM handbook.
- `School_OS_SaaS_Project_Management_Master_v0.2.xlsx` - updated PM workbook including future roadmap and attendance-device architecture decision.
- `School_OS_Codex_Environment_Guide_v0.2.pdf` - human-readable Codex/environment handoff generated from this pack.

Version note

The original master PDF/DOCX is v0.1 and remains a valid historical baseline. The Markdown master in this pack is v0.2 and adds the future attendance capture architecture. When a conflict exists, the latest Markdown plus approved ADR/change log takes precedence.

Additional Codex/developer context

- `QUESTION_AND_DECISION_MAP.md` - compact discovery-question -> decision mapping, including Q50-Q100 and final SaaS choice.
 - `REPOSITORY_STRUCTURE.md` - recommended VS Code/repository layout.
 - `../CODEX_BOOTSTRAP_PROMPT.md` - optional prompt for starting a fresh Codex session.
-